

One neuron: sigmoid, cross-entropy, a boundary

Friday's line predicted a number. Most questions worth asking a model are yes-or-no: will this patient miss the appointment, is this email spam, is this digit even. One new function turns a score into a probability, one new loss makes the probability honest, and the loop from Friday does the rest.

Unit 2 · From scratch

80 minutes

Lab 8 · one neuron

The first activation function

Why not a line

The sigmoid

Cross-entropy

Training

Lab 8

Before Friday

PART 1 · 10 MINUTES

Why a straight line cannot say yes or no

Predicting 1.37 for "did they pass?" is not an answer.

1

Keep Friday's model and change only the label: $y = 1$ if the student passed, 0 if not. Fit a line and it will happily predict 1.37 for a student who revised a lot and -0.2 for one who did not. Neither is a probability, and a single very diligent student far to the right drags the whole line flat. We need an output that lives between 0 and 1 and a model that reads as *how confident am I that the answer is yes*.

The sigmoid: any number in, a probability out

The first activation function, introduced because we need it.

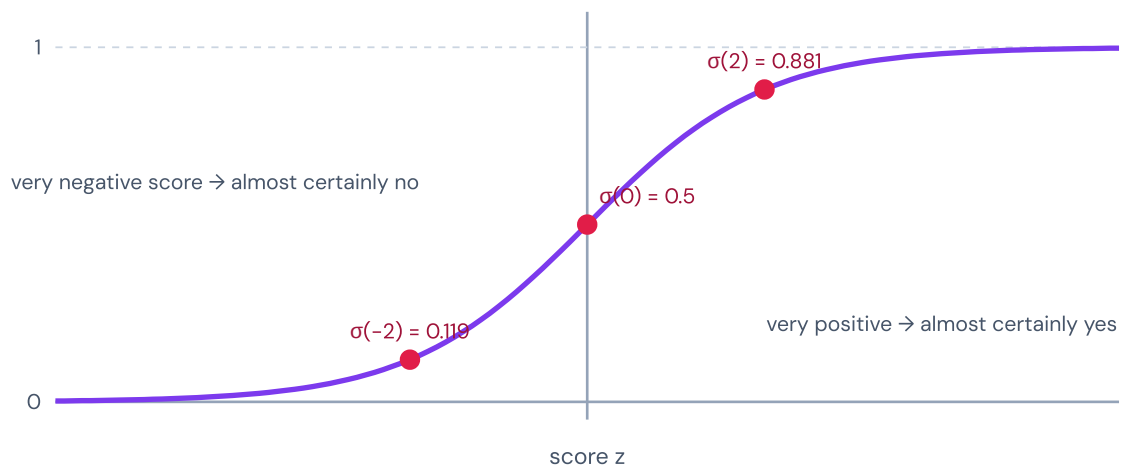
2

THE NEURON, IN TWO LINES

$$z = w \cdot x + b \quad p = \sigma(z) = 1 / (1 + e^{-z})$$

Compute a score exactly as on Friday — a weighted sum plus a bias. Then squash it.

THE SIGMOID NEVER REACHES 0 OR 1 — A NEURON IS NEVER COMPLETELY CERTAIN



z	-4	-2	0	2	4
$\sigma(z)$	0.018	0.119	0.500	0.881	0.982

Three facts to hold on to. **$\sigma(0) = 0.5$** : a score of zero is a coin toss. The curve is symmetric: $\sigma(-z) = 1 - \sigma(z)$. And its slope, which backpropagation will need, is written with its own output: **$\sigma'(z) = \sigma(z)(1 - \sigma(z))$** — largest at 0.25 in the middle and almost zero at the extremes. Remember that last fact; it explains a failure in session 19.

Cross-entropy: the loss that punishes confident mistakes

And why squared error is the wrong loss for a decision.

BINARY CROSS-ENTROPY

$$L = -\text{mean}(y \cdot \log p + (1 - y) \cdot \log(1 - p))$$

If the label is 1, the loss is $-\log p$: tiny when p is near 1, enormous when p is near 0. If the label is 0, the roles swap. Averaged over the data:

Label	Neuron says p	Loss $-\log(\dots)$	Reading
1	0.99	0.010	right and confident: almost free
1	0.5	0.693	a coin toss
1	0.01	4.605	confidently wrong: very expensive



0.693 is a number to remember

It is $\log 2$ — the loss of a model that says 50–50 about everything. A training run that sits at 0.693 has learned nothing. You will see it on Friday, from a neuron that is doing its honest best on a problem it cannot solve.

Why not keep squared error?

Squared error *works* through a sigmoid, in the sense that the code runs. The trouble is what happens when the neuron is badly wrong. Take a point with label 1 where the neuron says 0.001. The push each loss gives the score z :

the same comparison as Lab 8, task 5

PYTHON

```
p, y = 0.001, 1.0
push_bce = p - y                                # cross-entropy through a sigmoid
push_mse = 2 * (p - y) * p * (1 - p)            # squared error through a sigmoid
print(round(push_bce, 4), round(push_mse, 6))
```

output

OUTPUT

-0.999 -0.001996

THE FACTOR THAT KILLS LEARNING

$$\partial L / \partial z = p - y$$

Squared error drags in the sigmoid's slope $p(1 - p)$, which is almost zero exactly when the neuron is most wrong. Cross-entropy's log cancels that factor, and the gradient with respect to the score collapses to something beautifully short:

PART 4 · 12 MINUTES

Training it: Friday's loop, one line different

4

logistic regression from scratch – two features now

PYTHON

```
w, b, lr = np.zeros(2), 0.0, 0.5
for epoch in range(500):
    p = sigmoid(X @ w + b)      # the new line
    dz = p - y                  # from cross-entropy
    w -= lr * X.T @ dz / len(X) # same shape of update as Friday
    b -= lr * dz.mean()
```

Put it beside Friday's loop. The update is the same pattern: error times input, averaged, times the learning rate. What changed is one squash in the forward pass and the loss that produced `dz`. On Lab 8's forty points this reaches 100% training accuracy and a loss of about 0.012.

The decision boundary

The neuron says yes when $p > 0.5$, which is exactly when $z > 0$. So the boundary between the two answers is where $w_1x_1 + w_2x_2 + b = 0$ — and that is the equation of a **straight line**. Solve it for x_2 and you can draw it: $x_2 = -(w_1x_1 + b)/w_2$.

HOLD THIS FOR FRIDAY

A single neuron, however it is trained, can only ever separate its two classes with a straight line (a flat plane, with more features). Everything it learns is *where to put that line*. On Friday we meet four points where no line will do.

Three questions

The third is the one that matters for Friday.

Q1. A neuron outputs $p = 0.73$. What is its score z — positive or negative? Positive: $p > 0.5$ exactly when $z > 0$.

Q2. The label is 0 and the neuron says $p = 0.9$. What is dz ? $p - y = 0.9$. Positive, so the update lowers the score for this point — the right direction.

Q3. Double every weight and the bias. What happens to the boundary? Nothing: $w_1x_1 + w_2x_2 + b = 0$ has the same solutions after multiplying by 2. What changes is how *steep* the probabilities are near it — the neuron becomes more confident, not more correct. This is why, on separable data, the weights keep growing forever.

LAB 8 · 20 MINUTES IN CLASS

One neuron, from scratch

5

Lab 8 — `CSC4315_Lab08_One_Neuron`. You write the sigmoid, the loss, the three gradient lines and the boundary. Data and plotting are given.

Before Friday

1. **Finish Lab 8.**
2. **Without notes:** the sigmoid, its value at 0, the cross-entropy formula, and why $p - y$ is the gradient.
3. **On paper:** draw four points at the corners of a unit square, labelled 0, 1, 1, 0 going round. Try to draw one line that separates the 1s from the 0s.



Session 12 — Friday 16 October, 11:30

The perceptron and the wall it hit. A history that nearly ended the field — and the four points from your homework, demonstrated live.